APPLIED MACHINE LEARNING SERIES

From Exploratory Analysis to Production

Comparing Regression Models on the Data & AI Salaries Dataset and Deploying the Best Model as a Backend Service

A STEP-BY-STEP PRACTICAL GUIDE

DATASET	ds_salaries.csv · 2,584 unique records
TARGET VARIABLE	salary_in_usd (continuous)
FEATURES	8 predictors: year, experience, job title, location, size, remote ratio
TOOLS	Python · scikit-learn · XGBoost · FastAPI · Docker
SCOPE	15 steps from model comparison to a monitored API

Version 1.0 · November 2026

CONTENTS

Table of Contents

Roadmap and Objectives . . . 3

Roadmap at a Glance . . . 3

Deliverables . . . 3

Prerequisites . . . 4

How to Use This Guide . . . 4

What the EDA Tells Us About Modelling . . . 5

Part A · Comparing Regression Models . . . 6

Step 1 · Set Up the Environment . . . 6

Step 2 · Rebuild the Modelling Table . . . 6

Step 3 · Split the Data . . . 7

Step 4 · Choose the Evaluation Metrics . . . 7

Step 5 · Build the Preprocessing Pipeline . . . 7

Step 6 · Define the Candidate Models . . . 8

Step 7 · Cross-Validate and Compare . . . 9

Step 8 · Tune the Finalists . . . 10

Step 9 · Evaluate on the Test Set and Select . . . 11

Part B · Deploying the Best Model . . . 13

Step 10 · Refit and Serialise . . . 13

Step 11 · Build the API with FastAPI . . . 13

Step 12 · Test the Service . . . 15

Step 13 · Containerise with Docker . . . 15

Step 14 · Deploy . . . 16

Step 15 · Monitor and Maintain . . . 17

Pitfalls and Final Checklist . . . 18

Final Checklist . . . 18

Appendix A · Results Log . . . 19

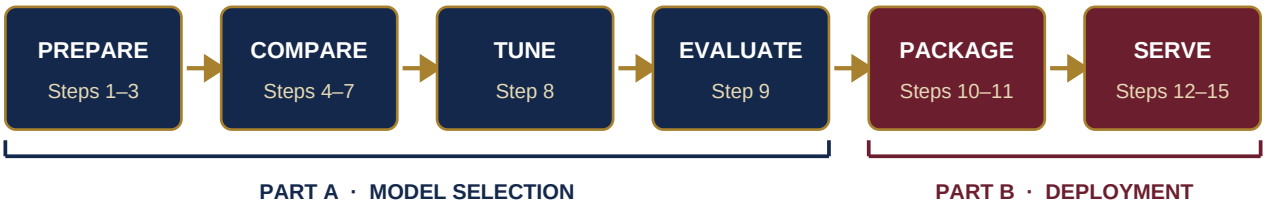
Appendix B · References . . . 19

CHAPTER 1

Roadmap and Objectives

Your exploratory data analysis (EDA) answered the question *"what is in this data?"* The next stage answers a different one: **which regression model predicts `salary_in_usd` most accurately and most reliably on records it has never seen, and how do we make that model available to a backend application?** This guide converts that goal into fifteen concrete, ordered steps. Every code listing was executed end-to-end on a dataset with the same schema as yours before being included here, so you can copy it into a notebook and adapt it.

The work is split into two parts. **Part A** (Steps 1–9) is a disciplined experiment: build a fair comparison between many regression algorithms and select one using evidence rather than preference. **Part B** (Steps 10–15) turns the winning model into a tested, containerised web service that any backend can call.



Roadmap at a Glance

Step	Action	Output
1	Set up the environment: virtual environment, pinned libraries, fixed seed	`requirements-dev.txt`
2	Rebuild the modelling table: reload raw CSV, undo EDA-only transformations, remove leakage	`X` (8 features), `y`
3	Split the data: 80 / 20 hold-out, stratified by experience level	`X_train`, `X_test`
4	Define metrics: RMSE, MAE, R^2 and a naive baseline	Metric contract
5	Build preprocessing inside a scikit-learn `Pipeline`	`make_preprocessor()`
6	Define the candidate models (11 regressors + dummy baseline)	`candidates` dict
7	Cross-validate every candidate (5 folds × 3 repeats) on the training set only	Comparison table
8	Tune the top 2–3 models; test a log-transformed target	`best_model`
9	Evaluate once on the test set; inspect residuals and importance; decide	Selected model + report
10	Refit on all rows; serialise pipeline and metadata	`.joblib`, `metadata.json`
11	Build the API with FastAPI and Pydantic validation	`app/` package
12	Test: unit tests and a notebook-versus-API consistency check	Green test suite
13	Containerise with Docker and pinned requirements	Docker image
14	Deploy to a cloud platform with HTTPS, authentication and CI/CD	Public endpoint
15	Monitor, log, detect drift and schedule retraining	Operations runbook

Deliverables

- A reproducible modelling notebook with a fixed random seed and pinned library versions.
- A comparison table of 11 regression algorithms plus a naive baseline, scored with cross-validation.

- One tuned, finally-tested model with residual and feature-importance diagnostics.
- A serialised pipeline (`.joblib`) and a `metadata.json` file describing exactly how it was built.
- A validated REST API (FastAPI) with automated tests, and a Docker image ready to deploy.

Prerequisites

- Python 3.10 or newer (3.11 recommended) and basic familiarity with pandas and scikit-learn.
- Your EDA notebook and the file `ds_salaries.csv`, kept in the same folder layout as in the EDA (`notebooks/` and `data/`).
- Docker Desktop for Step 13, and optionally a GitHub account for the CI/CD pipeline in Step 14.

How to Use This Guide

Listings are numbered and meant to be run **in order** as notebook cells: each one builds on the variables created by the previous ones. Paths such as `../data/ds_salaries.csv` assume that the modelling notebook lives in `notebooks/`, exactly like your EDA notebook. Four kinds of highlighted boxes appear throughout:

Label	Meaning
NOTE	Background information or an assumption worth knowing.
TIP	An optional improvement or a useful shortcut.
WARNING	A mistake that silently produces wrong or misleading results.
KEY	A rule or decision that the rest of the workflow depends on.

CHAPTER 2

What the EDA Tells Us About Modelling

A good modelling plan is built on what the EDA actually found. The table below connects specific results from your notebook to a concrete decision in the modelling stage. Several EDA conveniences (a datetime column, a salary band, grouped job titles) are helpful for exploration but would silently damage a predictive model, so they must be handled deliberately.

Finding in the EDA	Evidence in your notebook	Consequence for modelling
Target is right-skewed	Skew = 0.62; range $5,132 – $450,000	Test a `log1p` target (Step 8) but always report errors in USD.
Duplicates were removed	3,755 rows → 2,584 rows	Deduplicate **before** splitting; identical rows in train and test inflate scores.
Experience is the strongest signal	Median EN $61.9K → SE $148.8K → EX $189.6K	Encode as an ordered scale (0–3); expect it to lead feature importance.
High-cardinality categories	`job_title` 93, `employee_residence` 78, `company_location` 72 levels	Pool rare levels *inside* the pipeline using `min_frequency`.
Geographic imbalance	US = 73.3% of employee residences	Predictions for other countries are less reliable; inspect segment errors.
`work_year` **became datetime**	EDA Step 9 conversion	Revert to an integer. Tree models cannot extrapolate beyond the latest year (2023).
Derived columns exist	`salary_band`, `job_title_grouped`	Drop `salary_band` (built from the target = leakage). Replace `job_title_grouped` with in-pipeline pooling.
Unusual Hybrid result	Hybrid median $63,312 (n = 187) vs $130,000 – $137,500 for the others	Probably confounded by other variables; do not interpret causally.
Outliers	29 rows (1.1%) above $310,038	Keep (they are plausible executive pay); compare RMSE with MAE to see their influence.
No missing values	Missing = 0% for all columns	No imputer needed, but the API must **reject** incomplete requests.

WARNING · DATA LEAKAGE IN TWO PLACES

`salary_band` is computed from `salary_in_usd`; using it as a feature gives the model the answer. `job_title_grouped` was computed from the frequency of titles across the *entire* dataset, including future test rows. Both must be removed from the feature set; the pipeline in Step 5 rebuilds the grouping from the training data only.

NOTE · SET REALISTIC EXPECTATIONS

The dataset has no years of experience, industry, skills, education or city. These strongly affect pay, so there is a ceiling on achievable accuracy. Success is therefore measured **relative to the baseline** and judged by whether the typical error (MAE) is useful for your application, not by chasing a near-perfect R^2.

CHAPTER 3 · STEPS 1–9

Part A · Comparing Regression Models

Step 1 · Set Up the Environment

Work in an isolated environment so that results can be reproduced later and the same library versions can be reused for deployment. Create a new notebook (for example `notebooks/02_modeling.ipynb`) next to your EDA notebook.

LISTING 1 · ENVIRONMENT SETUP (TERMINAL)

```
python -m venv .venv
.venv\Scripts\activate            # macOS/Linux:  source .venv/bin/activate
pip install pandas numpy scikit-learn scipy matplotlib seaborn
pip install xgboost lightgbm joblib
pip freeze > requirements-dev.txt
```

Record the versions you used (`pip freeze`). The scikit-learn version in particular must match between training and serving (Step 10).

Step 2 · Rebuild the Modelling Table

Do not continue with the `df` that your EDA modified. Reload the raw file and re-apply only the cleaning decisions that help prediction. The table lists each EDA transformation and what to do with it.

EDA transformation	Keep?	Reason
Strip and upper-case text labels	Yes	Guarantees consistent category names.
`drop_duplicates()`	Yes	Done before splitting to prevent train/test leakage.
`category` **dtype**	No	Not required; scikit-learn encoders handle strings.
`remote_ratio` **mapped to text**	No	Keep the original 0 / 50 / 100; it is encoded as a categorical variable.
`work_year` **converted to datetime**	No	Keep as an integer so models can use it numerically.
`salary_band`	Remove	Created from the target (leakage).
`job_title_grouped`	Remove	Replaced by training-only pooling in the pipeline.

LISTING 2 · LOAD RAW DATA AND CREATE X AND Y

```
import numpy as np
import pandas as pd

RANDOM_STATE = 42
TARGET = "salary_in_usd"

# 1) Re-load the RAW file (not the df that you modified during the EDA)
df = pd.read_csv("../data/ds_salaries.csv")

# 2) Re-apply only the cleaning decisions you made in the EDA
text_cols = ["experience_level", "employment_type", "job_title",
             "employee_residence", "company_location", "company_size"]
for col in text_cols:
    df[col] = df[col].str.strip()
for col in text_cols:
    if col != "job_title":
        df[col] = df[col].str.upper()

df = df.drop_duplicates().reset_index(drop=True)   # 3,755 -> 2,584 rows
```

```
# 3) Separate features and target
X = df.drop(columns=[TARGET])     # work_year stays an integer (no datetime!)
y = df[TARGET]
print(X.shape, y.shape)           # expected: (2584, 8) (2584,)
```

Step 3 · Split the Data

Hold back 20% of the rows as a **test set**. It represents "the future": it is opened once, in Step 9, to produce the final honest score. All model selection and tuning happens on the remaining 80% using cross-validation. Stratifying by `experience_level` keeps the rare Executive group (about 3.7% of rows) represented in both parts.

LISTING 3 · TRAIN / TEST SPLIT

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=X["experience_level"],   # keep EN/MI/SE/EX proportions equal
)
print(len(X_train), len(X_test))      # about 2067 / 517
```

> **TIP · OPTIONAL STRESS TEST**
>
> Because `work_year` spans 2020–2023, you can additionally train on years ≤ 2022 and test on 2023. This time-aware split shows how the model behaves on newer data and is a useful sentence in a report, but keep the random stratified split as the main experiment.

Step 4 · Choose the Evaluation Metrics

Metric	Meaning	Why it is used here
RMSE (USD)	Square root of the mean squared error	**Primary metric.** Same unit as salary; penalises large mistakes, which matter for high salaries.
MAE (USD)	Mean absolute error	The "typical miss". Easy to explain; less affected by the 29 outliers.
R^2	Share of salary variance explained	Scale-free summary; 0 means no better than predicting the mean.
Baseline	`DummyRegressor` predicting the median	Any real model must beat this clearly, otherwise it has learned nothing.

If RMSE and MAE disagree on the ranking, the model with the lower RMSE is making fewer very large errors, while the lower-MAE model is better for the typical case. Decide which matters for your application and state it in advance.

Step 5 · Build the Preprocessing Pipeline

Models need numbers, so each column receives an encoding suited to its nature. Everything is wrapped in a `ColumnTransformer` inside a `Pipeline`. This has two major benefits: (1) statistics such as the list of rare categories are learned only from training folds, so there is no leakage; and (2) the exact same transformations are saved with the model and re-applied automatically in production.

Column(s)	Encoding	Reason
`experience_level`	Ordinal: EN=0, MI=1, SE=2, EX=3	Natural order; median salary rises with each level.
`work_year`	Numeric (scaled for linear models and KNN)	Small integer range, 2020–2023.
`employment_type`, `company_size`, `remote_ratio`	One-hot	Few unordered levels.
`job_title`, `employee_residence`, `company_location`	One-hot with `min_frequency=0.01`	Levels below 1% of training rows are pooled into an “infrequent” group; unseen values in production map there safely.

LISTING 4 · PREPROCESSING

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler

EXP_ORDER   = ["EN", "MI", "SE", "EX"]                # junior -> executive
numeric_cols = ["work_year"]
onehot_low   = ["employment_type", "company_size", "remote_ratio"]
onehot_high  = ["job_title", "employee_residence", "company_location"]

def make_preprocessor(scale: bool) -> ColumnTransformer:
    exp_enc = OrdinalEncoder(categories=[EXP_ORDER])
    if scale:                                         # linear models and KNN
        exp_enc = Pipeline([("ord", exp_enc), ("sc", StandardScaler())])
    return ColumnTransformer([
        ("exp", exp_enc, ["experience_level"]),
        ("num", StandardScaler() if scale else "passthrough", numeric_cols),
        ("cat_low", OneHotEncoder(handle_unknown="ignore",
                                  sparse_output=False), onehot_low),
        # rare categories (< 1 % of the TRAINING rows) are pooled automatically
        ("cat_high", OneHotEncoder(handle_unknown="infrequent_if_exist",
                                   min_frequency=0.01,
                                   sparse_output=False), onehot_high),
    ])
```

Step 6 · Define the Candidate Models

Compare algorithms from different families. A linear model shows how far simple additive effects go; trees and boosting capture interactions such as “senior role *and* US-based”. All candidates use the same preprocessing and the same cross-validation splits, so differences are due to the algorithm only.

Family	Models	What to look for
Baseline	Dummy (median)	The score to beat.
Linear	Linear Regression, Ridge, Lasso, ElasticNet	Fast, interpretable; limited with interactions. Ridge/Lasso add regularisation.
Instance-based	K-Nearest Neighbours	Sensitive to scaling and many one-hot columns.
Single tree	Decision Tree	Easy to explain; unstable and prone to overfitting.
Bagging	Random Forest	Strong, robust default; larger files and slower predictions.
Boosting	Gradient Boosting, HistGradientBoosting, XGBoost, LightGBM	Often most accurate on tabular data; need tuning.

LISTING 5 · CANDIDATE MODELS

```
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV, ElasticNetCV
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import (RandomForestRegressor, GradientBoostingRegressor,
                              HistGradientBoostingRegressor)
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

RS = RANDOM_STATE

def make_pipe(model, scale=False):
    return Pipeline([("pre", make_preprocessor(scale)), ("model", model)])

candidates = {
    "Dummy (median)":       make_pipe(DummyRegressor(strategy="median")),
    "Linear Regression":    make_pipe(LinearRegression(), scale=True),
    "Ridge (CV)":           make_pipe(RidgeCV(alphas=np.logspace(-2, 3, 30)), True),
    "Lasso (CV)":           make_pipe(LassoCV(cv=5, max_iter=20000,
                                              random_state=RS), True),
    "ElasticNet (CV)":      make_pipe(ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8], cv=5,
                                                   max_iter=20000,
                                                   random_state=RS), True),
    "KNN":                  make_pipe(KNeighborsRegressor(n_neighbors=15,
                                                          weights="distance"), True),
    "Decision Tree":        make_pipe(DecisionTreeRegressor(max_depth=8,
                                      min_samples_leaf=10, random_state=RS)),
    "Random Forest":        make_pipe(RandomForestRegressor(n_estimators=300,
                                      min_samples_leaf=3, n_jobs=-1,
                                      random_state=RS)),
    "Gradient Boosting":    make_pipe(GradientBoostingRegressor(random_state=RS)),
    "HistGradientBoosting": make_pipe(HistGradientBoostingRegressor(random_state=RS)),
    "XGBoost":              make_pipe(XGBRegressor(n_estimators=400,
                                      learning_rate=0.05, max_depth=4,
                                      subsample=0.8, colsample_bytree=0.8,
                                      n_jobs=-1, random_state=RS)),
    "LightGBM":             make_pipe(LGBMRegressor(n_estimators=400,
                                      learning_rate=0.05, num_leaves=15,
                                      subsample=0.8, subsample_freq=1,
                                      colsample_bytree=0.8, verbose=-1,
                                      n_jobs=-1, random_state=RS)),
}
```

*Optional additions: **CatBoost** (handles categorical text natively and often works well on this kind of data) and **SVR**. Add them to the dictionary in the same way.*

Step 7 · Cross-Validate and Compare

Each model is trained and scored 15 times (5 folds repeated 3 times with different shuffles) on the training set. The mean is a stable estimate of future performance and the standard deviation tells you how much the score depends on luck.

LISTING 6 · REPEATED CROSS-VALIDATION

```
from sklearn.model_selection import RepeatedKFold, cross_validate

cv = RepeatedKFold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
scoring = {"rmse": "neg_root_mean_squared_error",
           "mae":  "neg_mean_absolute_error",
           "r2":   "r2"}

rows = []
for name, model in candidates.items():
    s = cross_validate(model, X_train, y_train, cv=cv, scoring=scoring,
                       return_train_score=True, n_jobs=-1)
    rows.append({
        "Model":        name,
        "CV RMSE":      -s["test_rmse"].mean(),
```

```
        "RMSE std":     s["test_rmse"].std(),
        "CV MAE":       -s["test_mae"].mean(),
        "CV R2":        s["test_r2"].mean(),
        "Train R2":     s["train_r2"].mean(),
        "Fit time (s)": s["fit_time"].mean(),
    })

results = pd.DataFrame(rows).sort_values("CV RMSE").reset_index(drop=True)
results["Overfit gap"] = results["Train R2"] - results["CV R2"]
results.round(3)
```

HOW TO READ THE RESULTS TABLE

- **CV RMSE ± std:** lower is better. If two models differ by less than one standard deviation, treat them as tied.
- **Overfit gap** (`Train R2` − `CV R2`): a gap above roughly 0.2 means the model memorises the training data; prefer a smaller gap at similar accuracy.
- **Fit time:** relevant if you plan to retrain often.
- **One-standard-error rule:** among models within one standard deviation of the best, choose the simplest one.
- **Sanity check:** every serious model must beat `Dummy (median)`. If none does by a wide margin, revisit the features before tuning.

Step 8 · Tune the Finalists

Hyperparameters (tree depth, learning rate, number of trees and so on) change accuracy noticeably. Tune only the top two or three models from Step 7 with a randomised search, which explores many combinations more efficiently than a full grid. The search sees only the training data; the test set is still sealed.

LISTING 7 · RANDOMISED SEARCH (SHOWN FOR XGBOOST)

```
from scipy.stats import randint, uniform
from sklearn.model_selection import RandomizedSearchCV, KFold

param_dist = {
    "model__n_estimators":     randint(200, 900),
    "model__learning_rate":    uniform(0.01, 0.14),
    "model__max_depth":        randint(2, 8),
    "model__min_child_weight": randint(1, 20),
    "model__subsample":        uniform(0.6, 0.4),
    "model__colsample_bytree": uniform(0.6, 0.4),
    "model__reg_lambda":       uniform(0.5, 10),
}

search = RandomizedSearchCV(
    candidates["XGBoost"],                # use the winner of Step 7
    param_distributions=param_dist,
    n_iter=60,
    scoring="neg_root_mean_squared_error",
    cv=KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1, random_state=RANDOM_STATE, refit=True,
)
search.fit(X_train, y_train)              # the test set is NOT used here

print("Best CV RMSE:", round(-search.best_score_, 1))
print(search.best_params_)
best_model = search.best_estimator_
```

Next, test whether predicting **log-salary** helps. Because salaries are right-skewed, a log target can stabilise errors across low and high pay. The wrapper below trains on the log scale but returns predictions in dollars, so metrics remain comparable.

LISTING 8 · LOG-TARGET EXPERIMENT

```
from sklearn.base import clone
from sklearn.compose import TransformedTargetRegressor

log_model = TransformedTargetRegressor(
    regressor=clone(best_model), func=np.log1p, inverse_func=np.expm1)

s = cross_validate(log_model, X_train, y_train, cv=cv, scoring=scoring, n_jobs=-1)
print("log-target  RMSE:", -s["test_rmse"].mean(), "  R2:", s["test_r2"].mean())
```

Adopt the log target only if it improves CV RMSE or clearly improves the residual pattern. Otherwise keep the simpler raw-target model. For a more thorough search, the **Optuna** library is a popular alternative to `RandomizedSearchCV`.

Step 9 · Evaluate on the Test Set and Select

Now, and only now, score the chosen model on the sealed test set. This is the number you report. If it is far worse than the cross-validation score, something leaked or the split was unlucky; investigate before continuing.

LISTING 9 · FINAL TEST EVALUATION

```
from sklearn.metrics import (root_mean_squared_error, mean_absolute_error,
                             r2_score)

pred  = best_model.predict(X_test)           # first and ONLY use of the test set
rmse  = root_mean_squared_error(y_test, pred)
mae   = mean_absolute_error(y_test, pred)
r2    = r2_score(y_test, pred)
print(f"Test RMSE = ${rmse:,.0f} | MAE = ${mae:,.0f} | R2 = {r2:.3f}")

# Error by segment: where does the model struggle?
resid = y_test - pred
by_level = (X_test.assign(abs_err=np.abs(resid))
                  .groupby("experience_level")["abs_err"]
                  .agg(["count", "mean"]).round(0))
print(by_level)
```

LISTING 10 · DIAGNOSTICS: RESIDUALS AND PERMUTATION IMPORTANCE

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 2, figsize=(10, 4))

ax[0].scatter(pred, y_test, alpha=0.4)
lims = [0, max(pred.max(), y_test.max())]
ax[0].plot(lims, lims, "--", color="black")
ax[0].set(xlabel="Predicted salary (USD)", ylabel="Actual salary (USD)",
          title="Predicted vs actual")

ax[1].scatter(pred, resid, alpha=0.4)
ax[1].axhline(0, ls="--", color="black")
ax[1].set(xlabel="Predicted salary (USD)", ylabel="Residual (USD)",
          title="Residuals")
plt.tight_layout(); plt.show()

from sklearn.inspection import permutation_importance
imp = permutation_importance(best_model, X_test, y_test,
                             scoring="neg_root_mean_squared_error",
                             n_repeats=20, random_state=RANDOM_STATE, n_jobs=-1)
print(pd.Series(imp.importances_mean, index=X_test.columns)
        .sort_values(ascending=False).round(0))
```

WHAT TO CHECK IN THE DIAGNOSTICS

- **Predicted vs actual:** points should hug the dashed diagonal. A flattened cloud means the model shrinks everything toward the mean.

- **Residuals:** should scatter around zero with no funnel or curve. A funnel shows larger errors at higher salaries, a reason to try the log target.
- **Error by segment:** compare MAE for EN / MI / SE / EX and for US versus non-US. Large differences must be reported as a limitation.
- **Permutation importance:** confirms the model relies on sensible features. Expect `experience_level` and `job_title` near the top; a surprising leader deserves a leakage check.

Finally, choose the model with a transparent decision matrix. The weights below are a suggestion; adapt them to your project.

Criterion	Weight	Question asked
Accuracy	40%	Lowest test and CV RMSE; clearly better than the baseline?
Stability	20%	Small standard deviation across folds and a small overfit gap?
Interpretability	15%	Can you explain predictions to a non-technical reader?
Speed and size	15%	Prediction latency and file size acceptable for the API?
Maintainability	10%	Few extra dependencies; easy to retrain and deploy?

> **KEY · DECISION RULE**
> Select the model that is best or statistically tied for best on accuracy **and** acceptable on the other criteria. Write down the reason. A slightly less accurate but far simpler model is often the better engineering choice.

CHAPTER 4 · STEPS 10–15

Part B · Deploying the Best Model

Step 10 · Refit and Serialise

Model selection is finished, so train the chosen pipeline one last time on **all** 2,584 rows; more data usually gives a slightly better production model. The test-set scores from Step 9 remain the numbers you report. Save the **entire pipeline** (preprocessing and model together), never the bare model, plus a small metadata file for traceability.

LISTING 11 · SAVE THE FINAL PIPELINE AND ITS METADATA

```
import json, datetime as dt
import joblib, sklearn
from pathlib import Path

final_model = clone(best_model).fit(X, y)          # refit on ALL 2,584 rows

Path("../models").mkdir(exist_ok=True)
joblib.dump(final_model, "../models/salary_model_v1.joblib", compress=3)

metadata = {
    "model_version": "v1",
    "algorithm": type(best_model.named_steps["model"]).__name__,
    "trained_on_rows": int(len(X)),
    "trained_at": dt.datetime.now().isoformat(timespec="seconds"),
    "sklearn_version": sklearn.__version__,
    "features": list(X.columns),
    "test_rmse": round(float(rmse), 2),
    "test_mae": round(float(mae), 2),
    "test_r2": round(float(r2), 4),
}
Path("../models/metadata.json").write_text(json.dumps(metadata, indent=2))
```

WARNING · VERSION AND SECURITY NOTES

Pin versions: a `.joblib` file created with one scikit-learn version may fail or behave differently under another. Use the same version in `requirements.txt` as in your notebook. **Trust:** `joblib` and `pickle` files can execute code when loaded, so only load files that you created or that come from a trusted source.

Step 11 · Build the API with FastAPI

A web API lets any backend (Node.js, Django, Spring, a mobile app) request predictions over HTTP with JSON. FastAPI is a good fit: it validates input automatically, produces interactive documentation at `/docs`, and is fast. Use this project layout.

LISTING 12 · PROJECT STRUCTURE

```
salary-api/
├── app/
│   ├── __init__.py
│   ├── main.py              # FastAPI application
│   └── schemas.py           # request / response validation
├── models/
│   ├── salary_model_v1.joblib
│   └── metadata.json
├── tests/
│   └── test_api.py
├── requirements.txt         # pinned versions
├── Dockerfile
└── .dockerignore
```

`schemas.py` defines exactly which inputs are allowed. Anything else is rejected with HTTP 422 before it reaches the model, which protects you from typos and malformed data.

LISTING 13 · APP/SCHEMAS.PY

```
from typing import Literal
from pydantic import BaseModel, Field, field_validator


class SalaryRequest(BaseModel):
    work_year: int = Field(..., ge=2020, le=2035, examples=[2023])
    experience_level: Literal["EN", "MI", "SE", "EX"]
    employment_type: Literal["FT", "PT", "CT", "FL"]
    job_title: str = Field(..., min_length=2, examples=["Data Scientist"])
    employee_residence: str = Field(..., min_length=2, max_length=2)
    remote_ratio: Literal[0, 50, 100]
    company_location: str = Field(..., min_length=2, max_length=2)
    company_size: Literal["S", "M", "L"]

    @field_validator("employee_residence", "company_location")
    @classmethod
    def upper_country(cls, v: str) -> str:
        return v.strip().upper()

    @field_validator("job_title")
    @classmethod
    def clean_title(cls, v: str) -> str:
        return v.strip()


class SalaryResponse(BaseModel):
    predicted_salary_usd: float
    typical_error_usd: float
    model_version: str
```

`main.py` loads the model once at start-up and exposes a health check and a prediction endpoint. The response includes the model's typical error (the test MAE), so callers do not mistake a point estimate for certainty.

LISTING 14 · APP/MAIN.PY

```
import json
from contextlib import asynccontextmanager
from pathlib import Path

import joblib
import pandas as pd
from fastapi import FastAPI

from app.schemas import SalaryRequest, SalaryResponse

MODEL_DIR = Path(__file__).resolve().parent.parent / "models"


@asynccontextmanager
async def lifespan(app: FastAPI):
    # load ONCE at start-up, not on every request
    app.state.model = joblib.load(MODEL_DIR / "salary_model_v1.joblib")
    app.state.meta = json.loads((MODEL_DIR / "metadata.json").read_text())
    yield


app = FastAPI(title="Data & AI Salary Prediction API",
              version="1.0.0", lifespan=lifespan)


@app.get("/health")
def health():
    return {"status": "ok", "model_version": app.state.meta["model_version"]}


@app.post("/predict", response_model=SalaryResponse)
```

```
def predict(req: SalaryRequest):
    X = pd.DataFrame([req.model_dump()])         # same column names as training
    pred = float(app.state.model.predict(X)[0])
    return SalaryResponse(
        predicted_salary_usd=round(max(pred, 0.0), 2),
        typical_error_usd=app.state.meta["test_mae"],
        model_version=app.state.meta["model_version"],
    )
```

Step 12 · Test the Service

Automated tests prevent silent breakage when you change code or retrain. Cover three things: the service starts, valid requests succeed, and invalid requests are rejected.

LISTING 15 · TESTS/TEST_API.PY (RUN WITH: PYTEST)

```
from fastapi.testclient import TestClient
from app.main import app

VALID = {
    "work_year": 2023, "experience_level": "SE", "employment_type": "FT",
    "job_title": "Data Scientist", "employee_residence": "us",
    "remote_ratio": 100, "company_location": "US", "company_size": "M",
}

def test_health():
    with TestClient(app) as client:
        r = client.get("/health")
        assert r.status_code == 200 and r.json()["status"] == "ok"

def test_predict_ok():
    with TestClient(app) as client:
        r = client.post("/predict", json=VALID)
        assert r.status_code == 200
        assert 10_000 < r.json()["predicted_salary_usd"] < 600_000

def test_invalid_level_rejected():
    with TestClient(app) as client:
        r = client.post("/predict", json={**VALID, "experience_level": "XX"})
        assert r.status_code == 422
```

> **TIP · THE CONSISTENCY TEST THAT MATTERS MOST**
>
> Take one row from the test set, predict it in the notebook, send the same values to the API and confirm that both numbers match to two decimals. This single check proves that preprocessing in production is identical to training.

Step 13 · Containerise with Docker

A Docker image packages your code, model and exact library versions so that it runs identically on your laptop, a colleague's computer and a cloud server. List dependencies with pinned versions first.

LISTING 16 · REQUIREMENTS.TXT

```
fastapi==0.115.*
uvicorn[standard]==0.30.*
pydantic>=2.7,<3
pandas==2.2.*
numpy<2.1
scikit-learn==1.5.*     # MUST equal the version used for training
xgboost==2.1.*          # only if the winner is XGBoost
joblib==1.4.*
# dev only: pytest, httpx
```

LISTING 17 · DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app

# LightGBM needs OpenMP on slim images (remove if you do not use it)
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ app/
COPY models/ models/

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

LISTING 18 · RUN LOCALLY, BUILD, AND CALL THE API

```
# local run
uvicorn app.main:app --reload          # http://127.0.0.1:8000/docs

# docker
docker build -t salary-api:1.0 .
docker run -p 8000:8000 salary-api:1.0

# example call
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"work_year":2023,"experience_level":"SE","employment_type":"FT",
       "job_title":"Data Scientist","employee_residence":"US",
       "remote_ratio":100,"company_location":"US","company_size":"M"}'
```

Step 14 · Deploy

Choose the platform that matches your experience and budget. All options below can run the Docker image from Step 13; check each provider's current free tiers and pricing before deciding.

Option	Best for	Notes
Render, Railway, Fly.io	Fast first deployment, student projects	Connect a Git repository or Docker image; HTTPS provided.
Google Cloud Run	Pay-per-use, automatic scaling	Runs any container; scales to zero when idle.
AWS (App Runner, ECS Fargate)	Production inside an AWS environment	More configuration; integrates with IAM and monitoring.
Azure Container Apps	Microsoft-centred organisations	Managed containers with built-in scaling.
VPS with Docker and Nginx	Full control, fixed monthly cost	You manage updates, TLS certificates and security.

PRODUCTION CHECKLIST

- **HTTPS only,** and restrict CORS to your own front-end origins.
- **Authenticate** callers (API key or JWT) if the endpoint is public, and add rate limiting.
- Keep secrets in **environment variables**, never in the repository.
- Automate with **CI/CD** (for example GitHub Actions): run tests, build the image, push it, then deploy.
- Call the API from your main backend with a timeout and error handling, as shown below.

LISTING 19 · CALLING THE SERVICE FROM ANOTHER PYTHON BACKEND

```
import requests

payload = {"work_year": 2023, "experience_level": "SE", "employment_type": "FT",
           "job_title": "Data Scientist", "employee_residence": "US",
           "remote_ratio": 100, "company_location": "US", "company_size": "M"}

r = requests.post("https://your-host/predict", json=payload, timeout=5)
r.raise_for_status()
salary = r.json()["predicted_salary_usd"]
```

Alternative: *if your backend is already written in Python, you can load the* `.joblib` *file directly and skip the separate service. The API approach is preferable when several applications share the model, when the backend uses another language, or when you want to scale and update the model independently.*

Step 15 · Monitor and Maintain

A deployed model degrades silently as the real world changes. Plan its maintenance before launch.

Area	What to do
Operational health	Track latency, error rate and uptime; expose `/health`; alert on failures.
Logging	Log request fields, prediction and model version (no personal data) to analyse usage later.
Data drift	Compare incoming values with the training data: share of unseen job titles or countries, `work_year` beyond 2023, new employment types.
Performance drift	When real salaries become available, recompute RMSE and MAE on recent data and compare with the Step 9 values.
Retraining	Retrain on a schedule or when drift appears; save as `salary_model_v2.joblib`, test, then switch traffic.
Rollback	Keep the previous version deployable so you can revert quickly.
Documentation	Write a short model card: purpose, data, metrics, known limitations (US dominance, no experience-years feature).

WARNING · KNOWN LIMITATION: FUTURE YEARS

Tree-based models predict a constant for any `work_year` above the maximum seen in training (2023). The API accepts years up to 2035, so either retrain with new data each year or clearly communicate that predictions reflect 2023 salary levels.

CHAPTER 5

Pitfalls and Final Checklist

Pitfall	Typical symptom	Prevention
Fitting preprocessing on all data	Test score unrealistically high	Use a `Pipeline`; fit only on training folds.
Using `salary_band` as a feature	R^2 close to 1.0	Drop every column derived from the target.
Duplicates across train and test	Large gap between CV and real-world scores	Deduplicate before splitting.
Tuning on the test set	Optimistic final report	Touch the test set exactly once.
Judging by a single split	Ranking changes with the random seed	Use repeated cross-validation and report the standard deviation.
Training/serving mismatch	API result differs from the notebook	Save the whole pipeline; run the consistency test.
Library version drift	Load error or different predictions	Pin versions; reuse them in Docker.
Over-interpreting predictions	Treating an estimate as an exact salary	Show the typical error; document limitations.

Final Checklist

- ☐ Raw data reloaded; `salary_band`, datetime and grouped titles removed
- ☐ Duplicates removed before the split; test set stored and untouched
- ☐ All preprocessing is inside one `Pipeline`
- ☐ At least 10 models compared with repeated CV; baseline included
- ☐ Top models tuned; log-target experiment recorded
- ☐ Test score computed once; residuals and importance reviewed
- ☐ Decision matrix completed and the choice justified in writing
- ☐ Final pipeline refit on all rows; `.joblib` and `metadata.json` saved
- ☐ API validates input; unit tests and consistency test pass
- ☐ Docker image builds and runs; versions pinned
- ☐ HTTPS, authentication and CORS configured before going public
- ☐ Monitoring, drift checks and retraining plan documented

APPENDIX

Appendix A · Results Log

Copy the values from the table produced in Step 7 and record your decision for each model. This table can be pasted directly into a report.

Model	CV RMSE	± std	CV MAE	CV R^2	Overfit gap	Fit (s)	Decision
Dummy (median)							
Linear Regression							
Ridge (CV)							
Lasso (CV)							
ElasticNet (CV)							
KNN							
Decision Tree							
Random Forest							
Gradient Boosting							
HistGradientBoosting							
XGBoost							
LightGBM							

Final selected model: _____ **Reason:** _____

Test RMSE: _____ **Test MAE:** _____ **Test R^2:** _____

Appendix B · References

1 Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

2 Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. O'Reilly Media.

3 Hastie, T., Tibshirani, R. and Friedman, J. *The Elements of Statistical Learning*. Springer.

4 Kuhn, M. and Johnson, K. *Applied Predictive Modeling*. Springer.

5 Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD 2016*.

6 scikit-learn documentation, "Cross-validation", "Pipelines and composite estimators", "Model persistence": scikit-learn.org

7 FastAPI documentation: fastapi.tiangolo.com · Docker documentation: docs.docker.com